Representation and Abstraction is All You Need

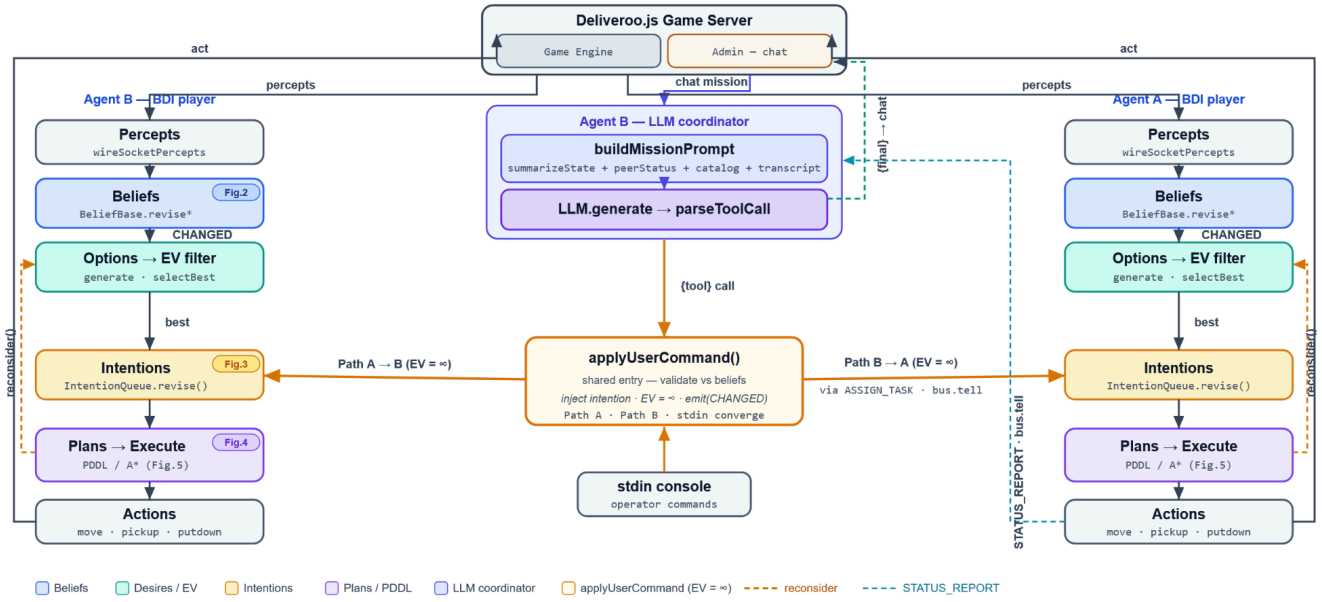
By Thomas Gunawisesa (265438) & Stephen Nganga (266153)

1. Introduction

We present a coordinated multi-agent system for the Deliveroo.js parcel-delivery game, comprising two BDI agents (Agent A and Agent B) that navigate, collect, and deliver parcels autonomously, with an LLM coordinator feature attached to Agent B that interprets natural-language mission requests issued by the professor through the game chat and drives an iterative tool-calling loop to solve them.

We show that the system's apparent complexity emerges not from monolithic design but from layering simple abstractions on low-level representations. We implement primitives such as Position, Tile, Parcel, Option, Intention and Command which ground every component, modular interfaces built on these primitives, a belief base with revision/query separation, an intention queue with pluggable commitment strategies, a plan selector with per-step reconsideration, and a shared command dispatcher. All of which compose into pipelines that produce autonomous behaviour, LLM-driven coordination, strategy adaptation, and planning as a service features. Each component demonstrates: a concrete data structure representation, a clean abstraction over it, and an emergent capability from their composition.

2. Architecture Overview



The autonomous BDI loop runs from percepts through belief revision, option generation, EV scoring, and intention revision into plan selection and execution. Actions flow back to the Deliveroo.js server, closing the sense-deliberate-act cycle. A reconsideration signal from the execution layer can break the cycle mid-plan, feeding back into option generation so the agent can preempt a failing or suboptimal intention. Both agents run this loop, Agent A and Agent B are identical BDI players that sense, move, pick up, and deliver parcels.

Agent B extends this base with an LLM coordinator feature. The professor issues special missions as plain-text messages in the game chat. The coordinator detects these messages (distinguishing them from typed coordination JSON by the absence of a protocol field), enqueues them in a FIFO queue so two missions never interleave, and solves each one with an iterative tool-calling loop: the LLM is prompted with the system rules, the mission text, a catalog of available tools, a fresh observation of the world state, and a trimmed transcript of steps taken so far. The LLM replies with exactly one JSON object per turn, either a tool call to execute or a final summary that ends the loop. Tool calls either act on Agent B itself (move, pickup, deliver, set_policy, add_constraint, pause, resume, wait) or delegate to Agent A via assign_to_teammate. Each tool returns an observation string that is fed back to the LLM, enabling it to adapt its plan dynamically across turns. A step budget bounds runaway loops.

Both command paths, local tool calls on Agent B and delegated commands to Agent A, converge on the same entry point. applyUserCommand() validates commands against current beliefs and injects them into the intention queue with infinite EV, bypassing the standard EV filter. This convergence is a deliberate design choice: a single dispatch function ensures that validation, belief checks, and intention injection are implemented once, regardless of whether the command originated from the LLM acting on itself or delegating to its teammate.

A PDDL planner plugs into the execution layer transparently. When enabled, the movement plan delegates to an external solver; when the solver is unreachable or returns an invalid plan, local A* takes over immediately. The rest of the system never knows which planner was used.

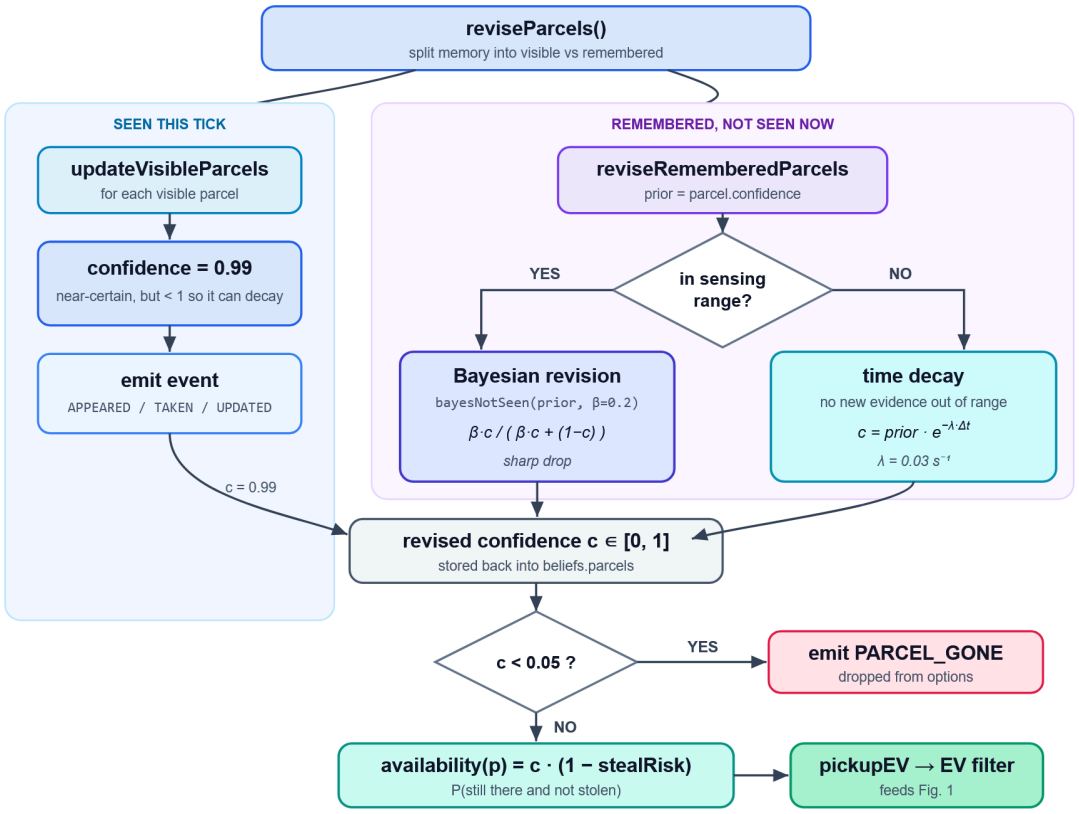
3. Representation

Every component in the system reads and writes six core types: Position (a grid coordinate), Tile (a map cell), Parcel (a sensed or remembered parcel with confidence and timestamp), Option (a candidate desire), Intention (a committed option with lifecycle metadata), and Command (a structured instruction from an LLM or human). Two additional types support the coordinator: Tool (a named capability in the LLM's catalog with an argument shape, description, and execute function) and ToolCall (one parsed reply from the iterative loop, either a tool invocation or a final summary). No module invents its own private data format; the belief base stores Tiles and Parcels, the option generator produces Options, the intention queue wraps Options in Intentions, the plan executor reads Intentions, the LLM adapter produces Commands, and the coordinator's tool catalog produces Tools that return observations consumed by the loop.

The BeliefBase is the central state container, a thin wrapper over two sibling modules. Revision functions mutate state and emit semantic events; query functions are pure and read-only. This separation means any module, e.g. pathfinding, EV scoring, the PDDL problem builder, can ask "is this tile walkable?" without touching state or triggering side effects.

4. BDI Architecture

4.1. Belief Revision



Parcel beliefs are never static. Every sensing tick, reviseParcels splits the parcel memory into two branches: parcels visible this tick get confidence 0.99 and a fresh lastSeenAt timestamp, while remembered parcels that were NOT seen diverge based on why they disappeared.

When a parcel is within sensing range but unseen, the evidence contradicts the belief that it still exists. Bayesian inference resolves the contradiction by computing

P(H | not Seen) = (sensorMissProbability * prior) / (sensorMissProbability * prior + (1 - prior))

When a parcel is outside sensing range, there is no contradiction, just absence of evidence. Confidence erodes gradually via exponential time decay:

prior x e^(-lambda x age), where lambda = 0.03 per second.

This distinction between revision and update prevents two failure modes: chasing ghost parcels that were picked up, and forgetting real parcels that simply moved out of view.

The final availability score = confidence * (1 - stealRisk). It feeds into pickupEV so that uncertain or contested parcels are automatically discounted in the option ranking.

4.2. Option Generation and EV Scoring

Each deliberation cycle produces a set of candidate desires and scores them with a single comparable utility number. The generator creates a pickup option for every visible free parcel, a

delivery option when the agent is carrying, and an explore fallback when nothing else is available. The filter then scores each option with expected value:

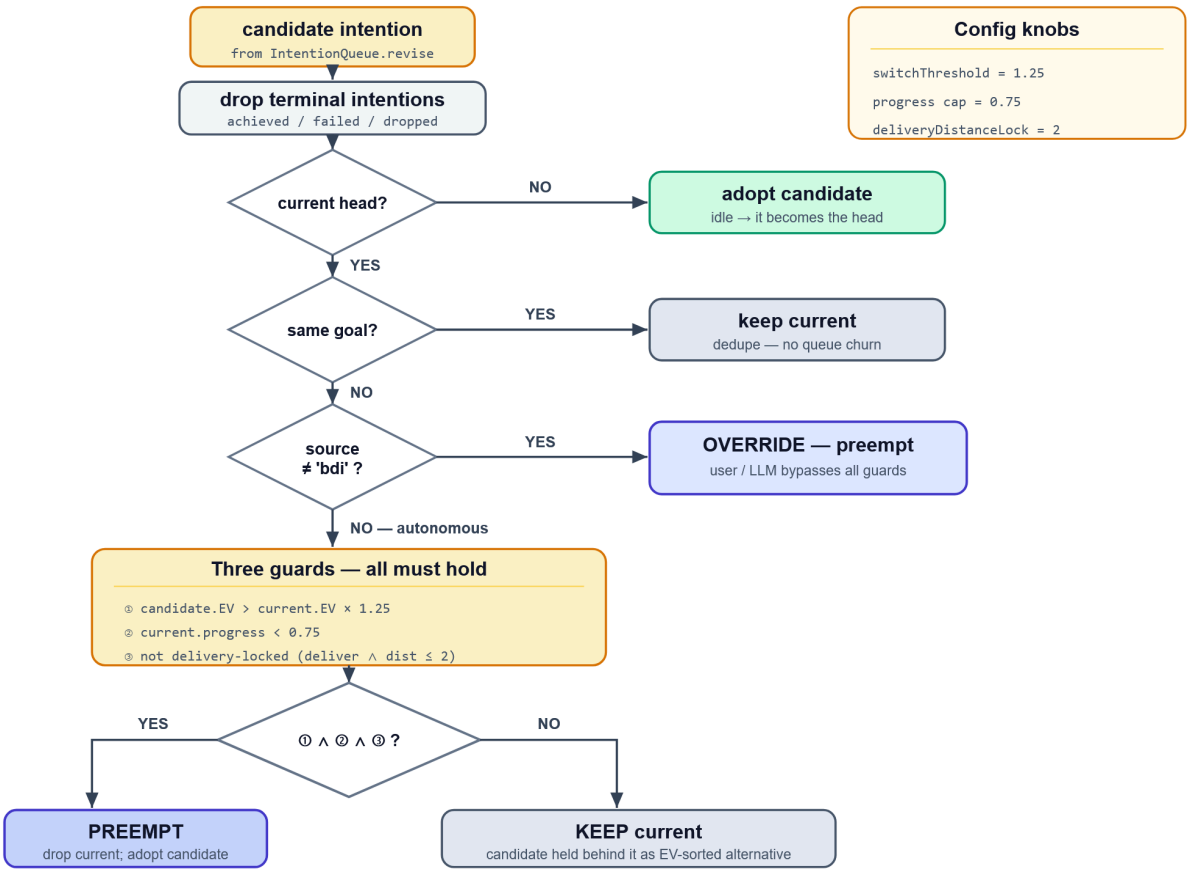
pickupEV = P(available) * decayedReward - travelCost

deliverEV = reward - distance + capacityPressureBonus.

Two design decisions in the EV calculation are worth noting:

1. Travel cost uses memoizedTravelCost, which runs A* over the belief map rather than Manhattan distance, so parcels behind walls are not deceptively cheap. The cost function is memoized per deliberation pass, meaning the shared "agent to nearest delivery zone" leg is computed once across all pickup options.
2. When the agent is already carrying parcels, pickupEV charges only the detour cost (the difference between going via the parcel and going straight to the zone), not the full fetch-and-deliver trip. This makes a nearby on-the-way parcel beat delivery, while a far off-path one does not, producing natural batching behaviour without an explicit heuristic.

4.3. Intention Revision



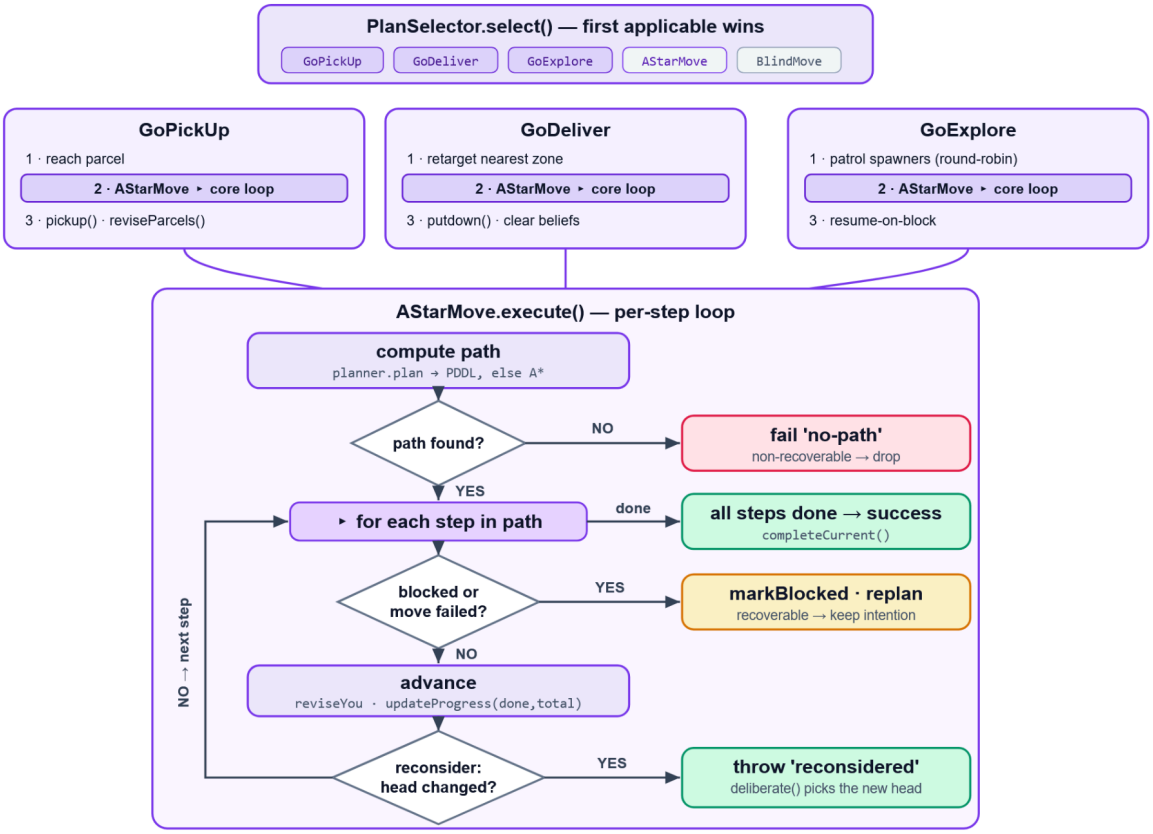
Intention revision applies three guards before allowing autonomous deliberation to abandon the current intention. A candidate must be clearly better (EV > current * 1.25), the current intention must not be nearly complete (progress < 0.75), and no delivery lock must be active (not within 2 steps of a delivery zone while carrying). Only when all three hold does the current intention get dropped in favour of the candidate.

Two mechanisms prevent pathologies that would otherwise arise:

1. The "goalIdentity" function produces a canonical key for each intention based on its logical goal (parcel id for pickups, sorted parcel ids for deliveries, coordinates for moves), deliberately ignoring volatile fields like reward and confidence.
2. The "samePredicate" function uses this key to recognise that two options from different ticks pursue the same goal, preventing the queue from stacking near-duplicate intentions. When the current intention is retained, the candidate is inserted into an EV-sorted alternative list behind it, so if the current intention fails, the next best option is immediately available.

Commands from outside the BDI loop (source = 'user' or 'llm') bypass all three guards entirely. A human typing "go to (10, 5)" mid-delivery or Agent B forwarding a mission request always preempts immediately. This ensures explicit instructions are never ignored by hysteresis or sunk-cost logic designed only to prevent autonomous thrashing.

4.4 Plan Execution



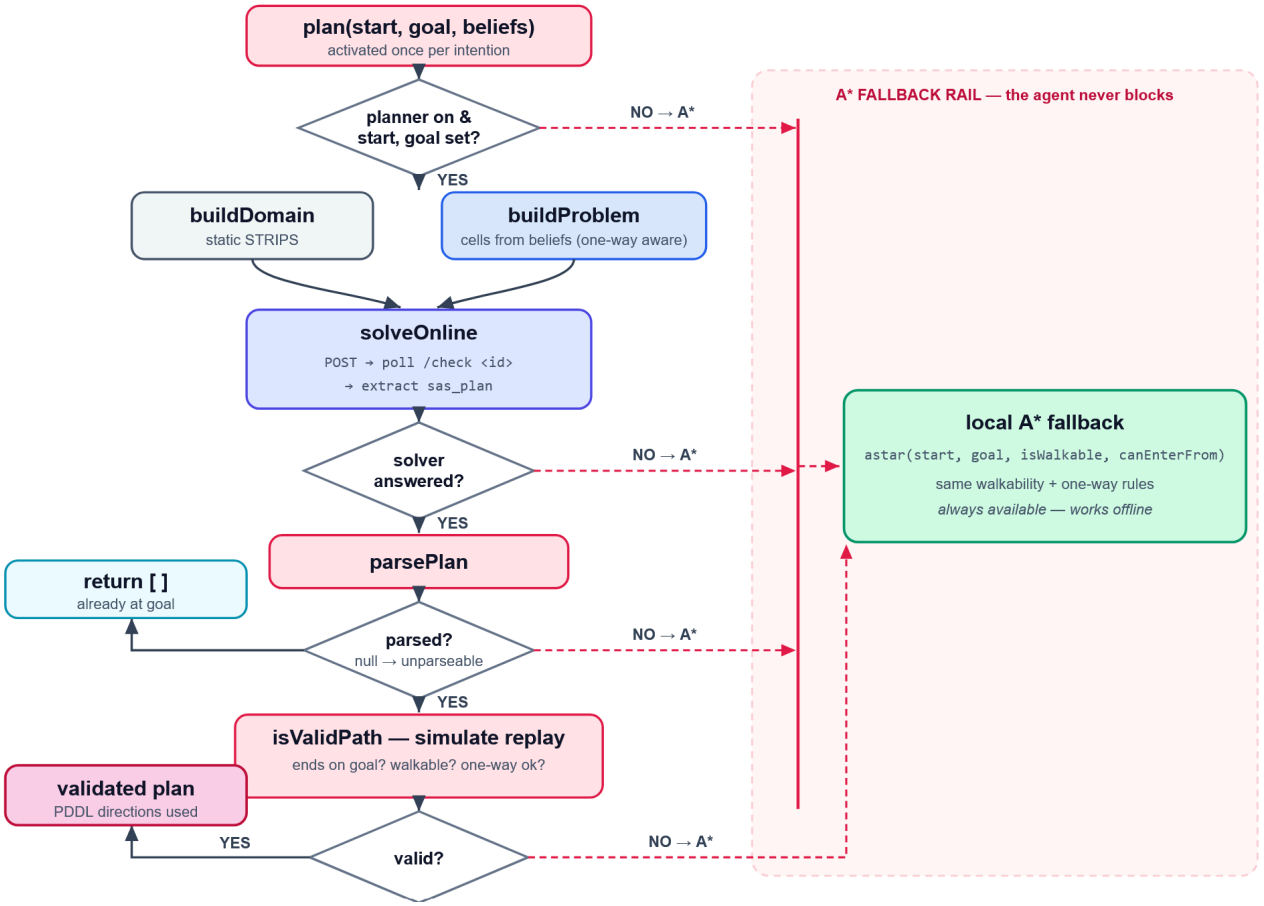
The PlanSelector tries plans in priority order: GoPickUp, GoDeliver, GoExplore, AStarMove, BlindMove. The first plan whose isApplicable returns true wins. GoPickUp and GoDeliver are compound plans that delegate navigation to AStarMove, then perform the pickup or putdown action and revise beliefs from the result. GoExplore patrols parcel spawner tiles in a round-robin cycle, also delegating movement to AStarMove, and skips tiles that are transiently blocked by other agents.

The AStarMove execution loop is where reconsideration happens. After each step, the "reconsider" callback runs a full deliberation cycle. If a better option emerges, the plan throws a PlanFailure tagged 'reconsidered', and the BDI loop continues to the next cycle with the new intention. This is open-minded commitment: the agent is committed to its plan but reconsiders after every single action, so preemption is immediate rather than deferred to the next percept.

The BDI loop itself iterates inside a single invocation rather than waiting for each CHANGED event. After a pickup completes, beliefs are revised locally (the parcel is now carriedBy the agent), but CHANGED only fires when the next server percept arrives, potentially 500ms later. Without the cycle loop, the agent would stand still between pickup and delivery. With it, pickup to delivery happens in one burst using already-updated local beliefs. A MAX_BDI_CYCLES cap of 100 prevents pathological cases where the loop would spin indefinitely on rapidly-completing explore legs with no parcels on the map.

A consecutive-failure counter prevents the agent from getting stuck on narrow maps where agents collide and cannot reroute. Every plan failure, recoverable or not, increments the counter. When it reaches a configurable threshold (default 3), the current intention is dropped and the agent enters a 1500ms cooldown matching the transient-block TTL, giving blocked tiles time to clear before re-deliberation picks an alternative goal. The counter resets on successful intention completion or when a plan is preempted by reconsideration.

5. PDDL Integration



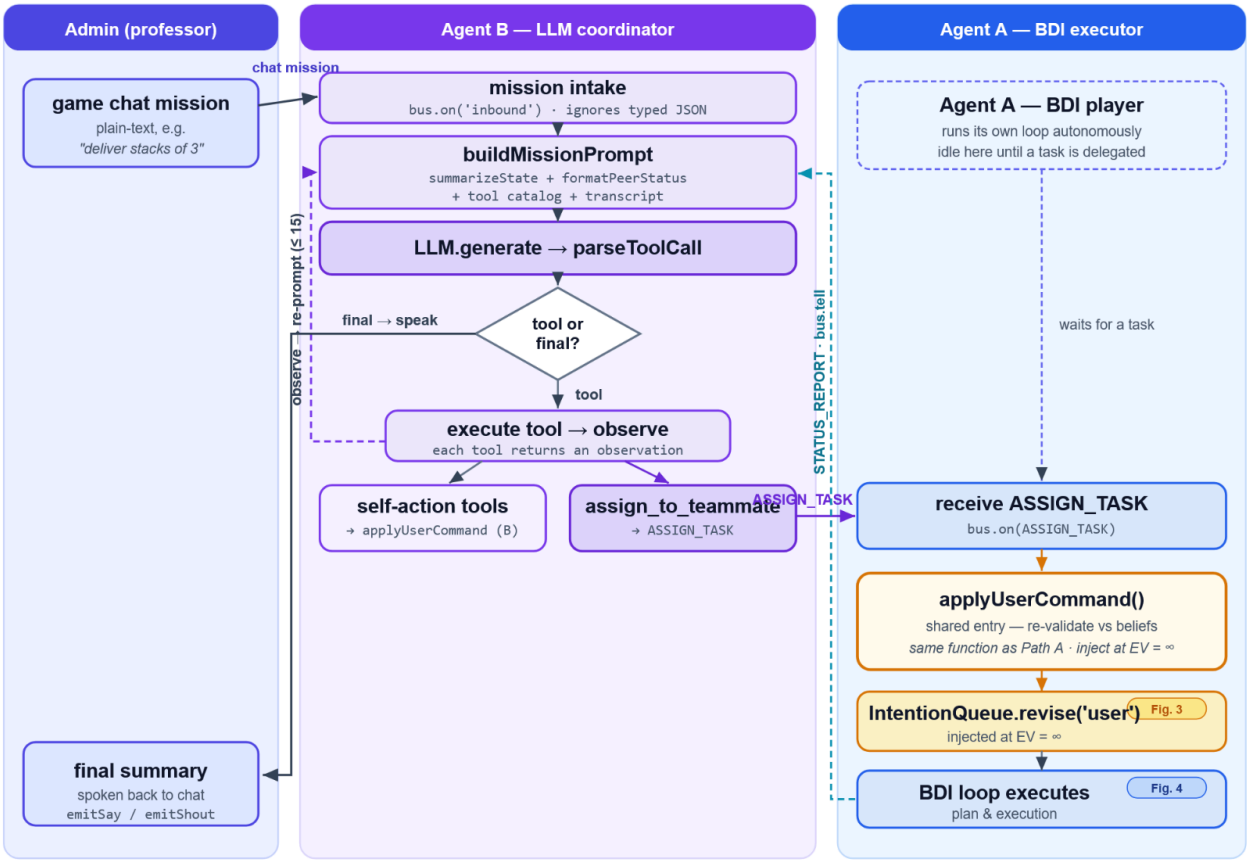
We model the spatial sub-problem of navigating to a target tile in PDDL. The domain is static: a single move action with precondition (at ?from) and (move-dir ?from ?to ?d), and effect (not (at ?from)) and (at ?to). The problem is generated dynamically from the belief base: every known walkable tile becomes a cell object, and adjacency facts encode which steps are legal, respecting walls and one-way tiles. The agent's current position is the initial state; the target tile is the goal.

The ExternalPlanner orchestrates the full cycle: build the problem, POST to the online solver, parse the returned plan into move directions, and validate the path against current beliefs by simulating each step and checking walkability and one-way constraints. Every failure path (network error,

timeout, unparseable response, invalid path) falls back to local A*. The agent never blocks and never produces an invalid move.

An issue emerged in testing: the solver takes 3 to 5 seconds per request, during which the agent stands still and carried parcels continue to decay. The final integration computes A* instantly as a fallback and races the solver against a 3-second timeout. If the solver wins, its validated plan is used; if it times out, A* is already ready. For competitive play where reward decay is aggressive, usePlanner can be set to false to skip the solver entirely, since A* produces equivalent paths with zero latency.

6. LLM Agent and Coordination



Agent B is a full BDI player, it senses, moves, picks up, and delivers parcels exactly like Agent A, with an LLM coordinator feature attached on top. The coordinator reads B's own beliefs, intentions, and message bus, adding three behaviours: team awareness, mission intake, and an iterative tool-calling loop.

The Admin issues special missions as plain-text messages in the game chat. The coordinator's inbound handler distinguishes these from typed coordination messages: anything without a protocol field is treated as a mission and enqueued in a FIFO queue so two missions never interleave their tool loops. Each mission is solved by an iterative loop with a configurable step budget (default 15): the LLM is prompted each turn with the system rules, the mission text, the tool catalog, a fresh observation of the world state, and a trimmed transcript of the last four steps taken. The LLM replies with exactly one JSON object, either a tool invocation or a final summary. Tool calls are executed against B's own BDI or delegated to Agent A; each returns an observation string fed back to the LLM. Invalid replies and tool errors become corrective transcript entries so the model can adapt within the

remaining budget rather than crashing. When the LLM produces a final summary, it is spoken back into the game chat.

Both command paths, local tools acting on Agent B and delegated commands to Agent A, converge on the same entry point. applyUserCommand() validates commands against current beliefs and injects them into the intention queue with infinite EV, bypassing the standard EV filter. This convergence is the key architectural insight: validation, belief checks, and intention injection are implemented once, regardless of command origin.

6.1. Tool Catalog

The coordinator exposes a catalog of tools that the LLM calls by name, one per turn. Each tool wraps one of Agent B's BDI capabilities or the team channel to Agent A:

1. observe: re-read the live world state and teammate status
2. move_to: walk Agent B to a coordinate
3. pickup: go pick up a specific parcel
4. deliver: deliver carried parcels to the nearest zone
5. set_policy: tune BDI strategy knobs (carryCapacity, lockedCapacity, agentStealRiskWeight, rewardDecayPerStep)
6. add_constraint: store behavioural constraints (avoidTiles, preferredDeliveryZones, blockedDeliveryZones, maxParcelReward)
7. assign_to_teammate: delegate one BDI command to Agent A over the team channel
8. send_chat: speak in the game chat
9. pause / resume / wait: control the BDI loop's running state

Local tools go through applyUserCommand, then emit a CHANGED event to wake B's own BDI loop. The assign_to_teammate tool sends an ASSIGN_TASK message to Agent A via bus.tell (directed emitSay), who applies it through the same applyUserCommand entry point into A's intention queue. Tool descriptions are engineered to guide the LLM toward correct usage: assign_to_teammate explains that Agent A is a full BDI agent that delivers autonomously after pickup, and that pause/resume of A should be done via set_policy with {paused: true/false} rather than explain commands.

6.2. Prompt Engineering

The system prompt distinguishes actions from strategy adjustments. "Deliver everything" must produce a deliver tool call (act now), not a set_policy call (change behaviour going forward). This distinction was not obvious to the LLM in early testing: "deliver stacks of exactly 3 parcels" was interpreted as a direct pickup-and-deliver sequence rather than a strategy change. The fix adds an explicit mapping in the prompt: missions that change how the agent should play map to set_policy or add_constraint, while missions that require immediate action map to move_to, pickup, or deliver.

The world state summary grounds each instruction with the agent's position, carried parcels, visible free parcels (with reward and confidence), delivery zones, current intention, and paused state. The teammate status line shows Agent A's position, carried count, current intention, and paused state. All coordinates are rounded to integers so the LLM never attempts to move to non-existent fractional tile positions.

6.3. Multi-Agent Coordination

Agent B and Agent A coordinate over the real game socket using directed messages (emitSay) rather than broadcasts (emitShout). Each agent discovers its peer through a one-time broadcast of a STATUS_REPORT, then switches to directed “tell” messages for all subsequent communication. This prevents the admin chat from being flooded with inter-agent protocol traffic, a problem observed in early testing when status reports and parcel claims were broadcast to all participants every sensing tick.

Three protocols coordinate belief-level behaviour: CLAIM_INTENTION records a peer's commitment to a parcel so option generation yields it, RELEASE_INTENTION clears the claim, and SHARE_BELIEFS merges a peer's parcel sightings into the local belief base. STATUS_REPORT is sent on a fixed interval (default 500ms) and carries position, carried parcels, current intention, and paused state, the last being essential for the LLM to detect when Agent A is frozen and needs a resume command. Peer claims auto-expire after a configurable TTL (default 10 seconds), so a disconnected peer cannot lock a parcel indefinitely. The wiring layer (wireTransport, wireBeliefCoordination) connects the socket to the bus and subscribes to these protocols in one call, keeping the agent's communication setup to a few lines at startup

6.4. Communication Layer

The MessageBus extends Node's EventEmitter with two directions: inbound socket messages are parsed and re-emitted as typed protocol events, and outbound bus events are serialised into emitShout or emitSay calls. The wiring layer connects the socket to the bus, routes typed messages to protocol handlers, and converts outbound bus events into the appropriate socket call based on whether a recipient is specified.

7. Testing

The BDI agent was validated against Challenge 1 scenarios. On maps with walls and one-way tiles, the agent successfully navigates using A* pathfinding that respects directional constraints. Belief revision was observed in practice: parcels leaving sensing range retain decaying confidence rather than vanishing instantly, and parcels within range that disappear trigger rapid Bayesian decay over 5 to 6 ticks. The agent autonomously cycles through pickup, delivery, and exploration without human intervention, and the reconsideration loop preempts suboptimal intentions mid-route when better options appear.

Two pathologies were identified and resolved during testing. First, on narrow maps where agents collide and cannot reroute, the BDI loop re-deliberates into the same blocked path indefinitely. A consecutive-failure counter (threshold 3, configurable) drops the intention and enters a 1500ms cooldown matching the transient-block TTL, giving blocked tiles time to clear. The GoExplore plan also now skips transiently blocked patrol targets. Second, the server reports positions as floating-point values (e.g. y: 14.4) but tiles are keyed by integers, so A* could never path to fractional coordinates. Math.round is now applied at all boundaries where server data enters the planning system.

The LLM coordinator was validated against Challenge 2 scenarios through live testing with an admin user sending missions via the game chat.

1. Atomic missions: General knowledge questions produce a final summary without invoking action tools. Move-to-coordinate missions inject high-priority intentions and the agent navigates to the target. A "stop moving" mission revealed the need for explicit pause, resume, and wait tools, plus exposing the paused state in the world summary so the LLM can detect and reverse it.
2. Strategy adaptation: "Deliver stacks of exactly 3 parcels" required two fixes: the set_policy tool description was updated with exact parameter names after the LLM guessed incorrectly, and the option generator was modified to suppress the delivery option when lockedCapacity is set. Three additional constraint types were implemented: delivery zone whitelisting/blacklisting, tile avoidance, and reward threshold filtering.
3. Coordination: Delegation from B to A was verified end-to-end. A peer discovery deadlock (neither agent sent a status report first) was fixed with a one-time broadcast before switching to directed messages. The LLM's tendency to send parcel ids A could not see was resolved by guiding it toward move-based delegation. Pause and resume of A through the team channel was verified, with A's status reports carrying the paused state so the LLM can detect and reverse it.

PDDL integration was demonstrated with the online solver producing valid plans on various maps. The A* fallback was tested by disabling the planner and by observing automatic fallback when the solver times out.

8. Conclusion

The system demonstrates that complex autonomous behaviour can emerge from layering simple abstractions on robust representations. Primitive representations ground every component. Modular interfaces compose into pipelines: belief revision produces confidence values that feed EV scoring, EV scoring produces selections that feed intention revision, intention revision produces commitments that drive plan execution, and plan execution produces actions that change the world, closing the loop.

Three architectural decisions proved especially valuable. The shared command dispatcher (applyUserCommand) unified LLM tool calls and delegated teammate commands through a single validation path, making Agent B both a player and a coordinator without duplicating logic. The iterative tool-calling loop replaced one-shot translation with a cycle of observe–act–adapt, letting the LLM respond to changing world state across multiple turns bounded by a step budget. Goal identity prevented intention queue thrashing by recognising stable goals across deliberation cycles.

Not all components reached their full potential. Level 3 coordination missions (multi-agent synchronized movement, parcel hand-off) have the transport layer in place but lack dedicated tools. Crates are not handled by the PDDL domain or the plan library. The PaaS solver latency (3 to 5 seconds per request) makes it impractical for competitive play with aggressive reward decay. Future work could extend the tool catalog for Level 3 patterns, model crate pushing in PDDL, and replace the round-robin exploration with a heatmap-driven patrol that prioritises high-spawn-probability tiles.